

### 1. Example of Use of struct (Slide 7)

```
#include <iostream>

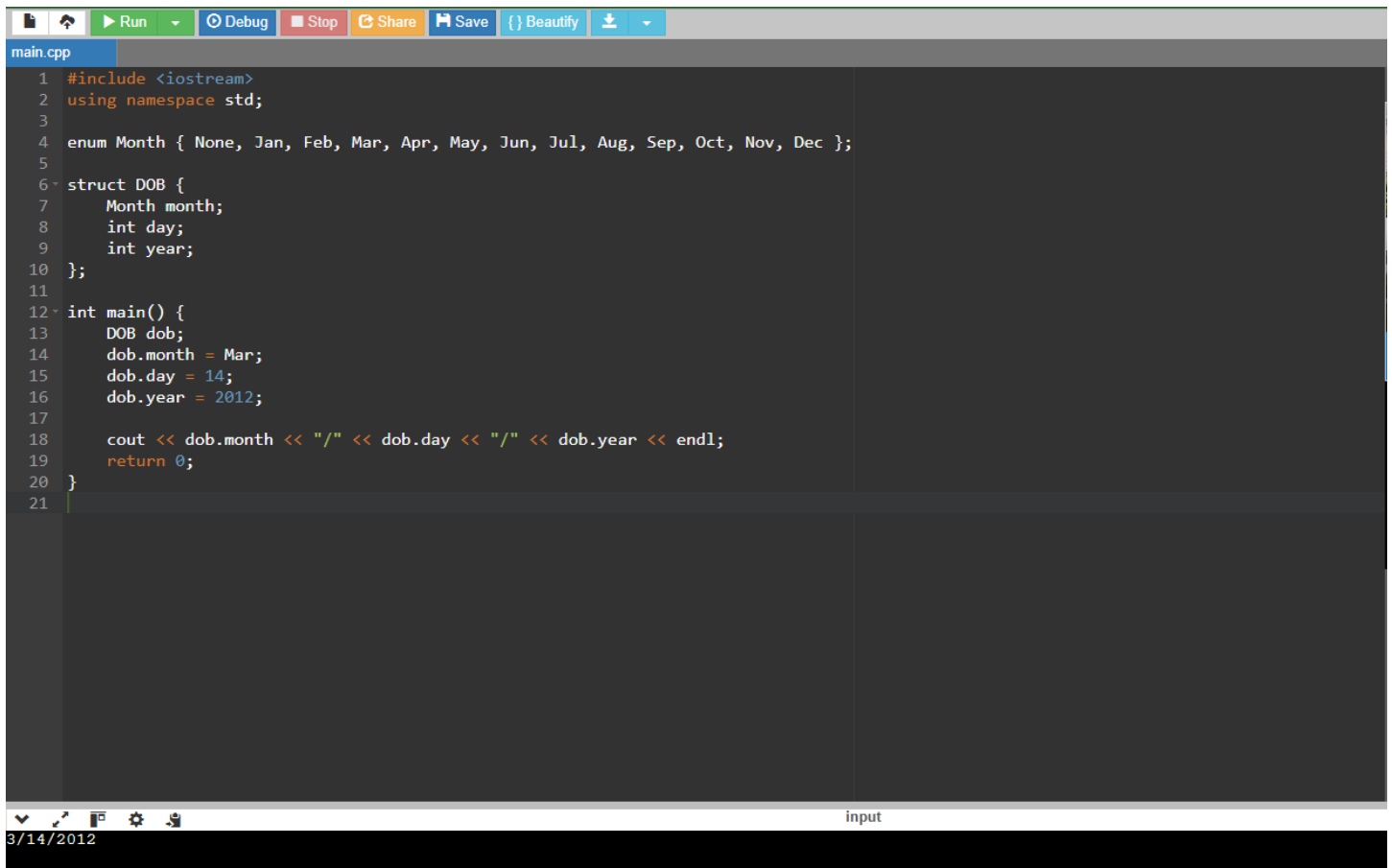
using namespace std;

enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };

struct DOB {
    Month month;
    int day;
    int year;
};

int main() {
    DOB dob;
    dob.month = Mar;
    dob.day = 14;
    dob.year = 2012;

    cout <<< dob.month <<< "/" <<< dob.day <<< "/" <<< dob.year <<< endl;
    return 0;
}
```



```
1 #include <iostream>
2 using namespace std;
3
4 enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
5
6 struct DOB {
7     Month month;
8     int day;
9     int year;
10 };
11
12 int main() {
13     DOB dob;
14     dob.month = Mar;
15     dob.day = 14;
16     dob.year = 2012;
17
18     cout << dob.month << "/" << dob.day << "/" << dob.year << endl;
19     return 0;
20 }
21
```

3/14/2012

## 2. Example of Class Definition (Slide 10)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
```

```
class DOB {
```

```
public:
```

```
    Month month = None;
```

```
    int day = 0;
```

```
    int year = 0;
```

```
    void setMonth(Month m) { month = m; }
```

```
    void setDay(int d) { day = d; }
```

```
void setYear(int y) { year = y; }
```

```
string getDate() const {
```

```
    return to_string(month) + "/" + to_string(day) + "/" + to_string(year);
```

```
}
```

```
};
```

```
int main() {
```

```
    DOB dob;
```

```
    dob.setMonth(Mar);
```

```
    dob.setDay(14);
```

```
    dob.setYear(2012);
```

```
    cout <<< dob.getDate() <<< endl;
```

```
    return 0;
```

```
}
```

The screenshot shows a C++ IDE with a toolbar at the top containing icons for Run, Debug, Stop, Share, Save, Beautify, and a dropdown menu. The file name 'main.cpp' is displayed in the top-left corner. The code is as follows:

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
4
5 enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
6
7 class DOB {
8 public:
9     Month month = None;
10    int day = 0;
11    int year = 0;
12
13    void setMonth(Month m) { month = m; }
14    void setDay(int d) { day = d; }
15    void setYear(int y) { year = y; }
16
17    string getDate() const {
18        return to_string(month) + "/" + to_string(day) + "/" + to_string(year);
19    }
20 };
21
22 int main() {
23     DOB dob;
24     dob.setMonth(Mar);
25     dob.setDay(14);
26     dob.setYear(2012);
27
28     cout << dob.getDate() << endl;
29     return 0;
30 }
31
```

At the bottom of the IDE, there is a status bar with icons for window management, settings, and a terminal icon. The date '3/14/2012' is shown on the left, and the word 'input' is on the right.

### 3. Example of Overloading (Slide 13)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
```

```
class DOB {
```

```
public:
```

```
    Month month = None;
```

```
    int day = 0;
```

```
    int year = 0;
```

```
    DOB() {}
```

```
    DOB(Month m, int d, int y) {
```

```

    month = m;

    day = d;

    year = y;
}

void setMonth() { month = Jan; }

void setMonth(Month m) { month = m; }


void setDay() { day = 1; }

void setDay(int d) { day = d; }


void setYear(int y) { year = y; }


string getDate() const {
    return to_string(month) + "/" + to_string(day) + "/" + to_string(year);
}

};


int main() {
    DOB dob(Mar, 14, 2012), phd;

    cout <<< dob.getDate() <<< " - " <<< phd.getDate() <<< endl;


    phd.setMonth();

    phd.setDay();

    phd.setYear(1998);


    cout <<< dob.getDate() <<< " - " <<< phd.getDate() <<< endl;

    return 0;
}

```

```
main.cpp
10 int day = 0;
11 int year = 0;
12
13 DOB() {}
14 DOB(Month m, int d, int y) {
15     month = m;
16     day = d;
17     year = y;
18 }
19
20 void setMonth() { month = Jan; }
21 void setMonth(Month m) { month = m; }
22
23 void setDay() { day = 1; }
24 void setDay(int d) { day = d; }
25
26 void setYear(int y) { year = y; }
27
28 string getDate() const {
29     return to_string(month) + "/" + to_string(day) + "/" + to_string(year);
30 }
31 };
32
33 int main() {
34     DOB dob(Mar, 14, 2012), phd;
35     cout << dob.getDate() << " - " << phd.getDate() << endl;
36
37     phd.setMonth();
38     phd.setDay();
39     phd.setYear(1998);
40
41     cout << dob.getDate() << " - " << phd.getDate() << endl;
42     return 0;
43 }
44
```

input

```
3/14/2012 - 0/0/0
3/14/2012 - 1/1/1998
```

#### 4. Example of Encapsulation (Slides 18–20)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
```

```
class DOB {
```

```
private:
```

```
    int day = 0;
```

```
    int year = 0;
```

```
    int encode(Month m, int d, int y) const {
```

```
        int limits[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

```

    if (y % 4 == 0) limits[2] = 29;
    for (int i = 2, acc = limits[1]; i < 13; ++i) {
        acc += limits[i];
        limits[i] = acc;
    }
    if (m == None && d == 0) return 0;
    return limits[int(m) - 1] + d;
}

```

```

void decode(int dy, Month &m, int &d, int y) const {
    int limits[13] = {0, 31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
    if (y % 4 == 0) limits[2] = 29;
    for (int i = 2, acc = limits[1]; i < 13; ++i) {
        acc += limits[i];
        limits[i] = acc;
    }
    if (dy > limits[12]) {
        m = None;
        d = 0;
    } else {
        for (int i = 1; i < 12; ++i) {
            if (dy <= limits[i]) {
                m = Month(i);
                d = dy - limits[i - 1];
                break;
            }
        }
    }
}

```

public:

```

DOB() {}

DOB(Month m, int d, int y) {
    doy = encode(m, d, y);
    year = y;
}

void setMonth(Month m) {
    Month mon; int d;
    decode(doy, mon, d, year);
    doy = encode(m, d, year);
}

void setDay(int d) {
    Month m; int day;
    decode(doy, m, day, year);
    doy = encode(m, d, year);
}

void setYear(int y) { year = y; }

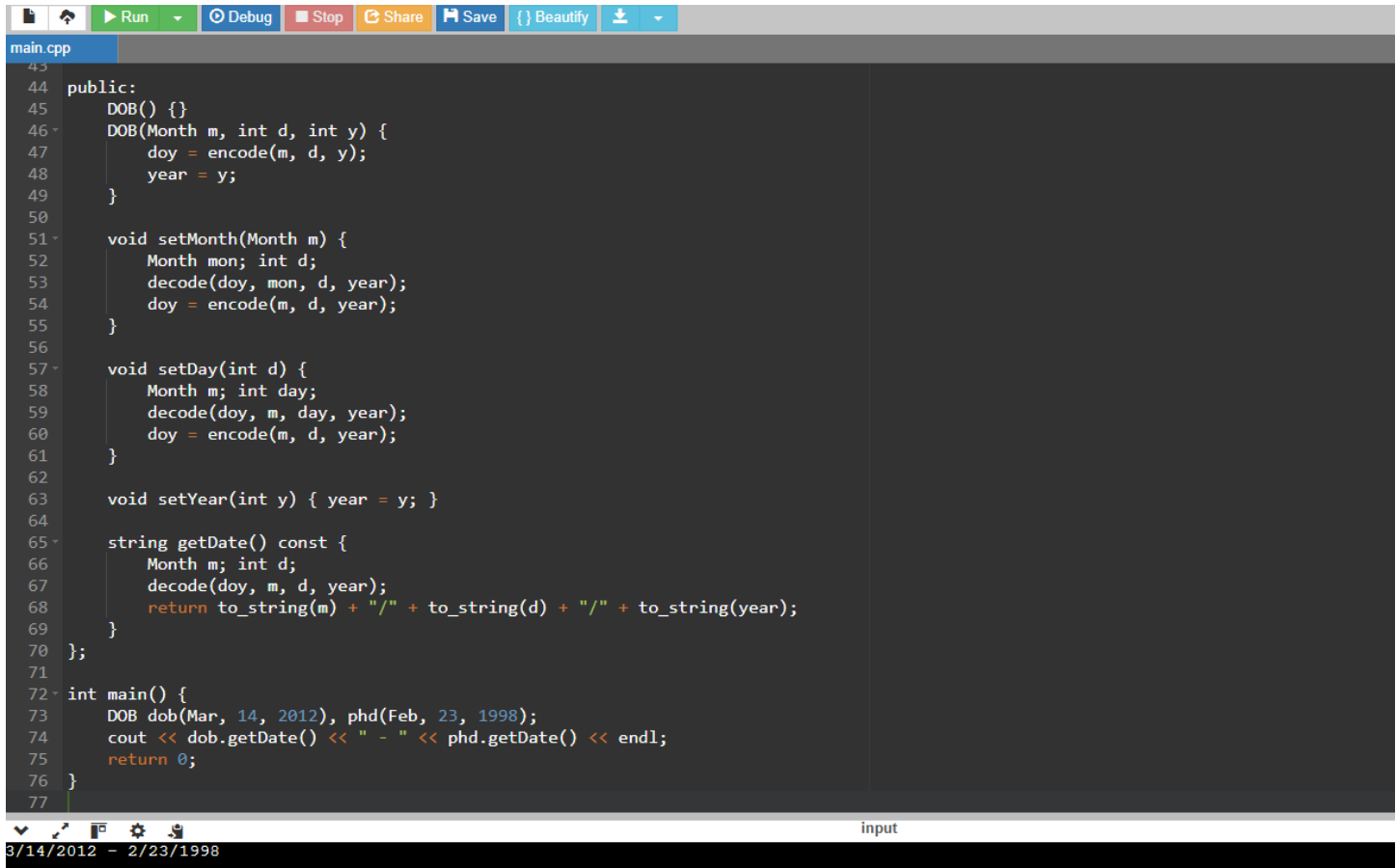
string getDate() const {
    Month m; int d;
    decode(doy, m, d, year);
    return to_string(m) + "/" + to_string(d) + "/" + to_string(year);
}

};

int main() {
    DOB dob(Mar, 14, 2012), phd(Feb, 23, 1998);
    cout <<< dob.getDate() <<< " - " <<< phd.getDate() <<< endl;
    return 0;
}

```

}



```
43
44 public:
45     DOB() {}
46     DOB(Month m, int d, int y) {
47         doy = encode(m, d, y);
48         year = y;
49     }
50
51     void setMonth(Month m) {
52         Month mon; int d;
53         decode(doy, mon, d, year);
54         doy = encode(m, d, year);
55     }
56
57     void setDay(int d) {
58         Month m; int day;
59         decode(doy, m, day, year);
60         doy = encode(m, d, year);
61     }
62
63     void setYear(int y) { year = y; }
64
65     string getDate() const {
66         Month m; int d;
67         decode(doy, m, d, year);
68         return to_string(m) + "/" + to_string(d) + "/" + to_string(year);
69     }
70 };
71
72 int main() {
73     DOB dob(Mar, 14, 2012), phd(Feb, 23, 1998);
74     cout << dob.getDate() << " - " << phd.getDate() << endl;
75     return 0;
76 }
77
```

3/14/2012 - 2/23/1998

## 5. Example of Inheritance and Polymorphism (Slide 26)

```
#include <iostream>
```

```
#include <string>
```

```
using namespace std;
```

```
enum Month { None, Jan, Feb, Mar, Apr, May, Jun, Jul, Aug, Sep, Oct, Nov, Dec };
```

```
class DOB {
```

```
protected:
```

```
    Month month = None;
```

```
    int day = 0;
```

```
    int year = 0;
```

public:

DOB() {}

DOB(Month m, int d, int y) {

month = m;

day = d;

year = y;

}

virtual string getDate() const {

return to\_string(month) + "/" + to\_string(day) + "/" + to\_string(year);

}

};

class HOB : public DOB {

private:

int hour = 0;

int mins = 0;

public:

HOB() {}

HOB(Month m, int d, int y, int h, int n) {

month = m;

day = d;

year = y;

hour = h;

mins = n;

}

string getDate() const override {

int h = (hour == 0) ? 12 : (hour < 13) ? hour : hour - 12;

string ampm = (hour < 12) ? "AM" : "PM";

```

        return to_string(month) + "/" + to_string(day) + "/" + to_string(year) +
            " " + to_string(h) + ":" + to_string(mins) + " " + ampm;
    }
};

int main() {
    DOB dob(Mar, 14, 2012);
    HOB hob(Feb, 23, 1998, 22, 30);

    cout <<< dob.getDate() <<< " - " <<< hob.getDate() <<< endl;

    return 0;
}

```

```

main.cpp
22     return to_string(month) + "/" + to_string(day) + "/" + to_string(year);
23 }
24 };
25
26 class HOB : public DOB {
27 private:
28     int hour = 0;
29     int mins = 0;
30
31 public:
32     HOB() {}
33     HOB(Month m, int d, int y, int h, int n) {
34         month = m;
35         day = d;
36         year = y;
37         hour = h;
38         mins = n;
39     }
40
41     string getDate() const override {
42         int h = (hour == 0) ? 12 : (hour < 13) ? hour : hour - 12;
43         string ampm = (hour < 12) ? "AM" : "PM";
44         return to_string(month) + "/" + to_string(day) + "/" + to_string(year) +
45             " " + to_string(h) + ":" + to_string(mins) + " " + ampm;
46     }
47 };
48
49 int main() {
50     DOB dob(Mar, 14, 2012);
51     HOB hob(Feb, 23, 1998, 22, 30);
52
53     cout << dob.getDate() << " - " << hob.getDate() << endl;
54     return 0;
55 }
56

```

input

3/14/2012 - 2/23/1998 10:30 PM